So we want to build a container orchestrator...

Aur Saraf, ACTC 2025



Who am I?

Aur, startup CTO / consultant

Wrote my first line of BASIC 27 years ago and fell in love

Run a programming Dojo every Wednesday 20:00 on Zoom

So we want to build a container orchestrator...

What are we building?

UX Desiderata; Technical Desiderata

How will we build it?

Architecture; Foundations; Knowing we finished

How do we start?

Planning; Starting small; Proof of concept

de · sid · er · a · tum (noun)

plural: desiderata

something that is needed or wanted.

UX Desiderata

Developer friendly

All Command&Control through an API (can build different clients)

REST API

API is uniform (learn one API, learn them all)

API is idempotent (do something twice == do it once)

User can be stateless because system manages all state

Build upon well-known container APIs

A good metaphor

Everything is Resources

Resources are JSON docs (can keep them in git, can generate them with code)

Store is single source of truth (reality != config? always fix reality)

Technical Desiderata

Stateless

Store is persisted, everything else is stateless and survives restart

High Availability

No single point of failure
Store is Consistent

Scalable

Every component can be sharded / replicated to scale to ∞

High Level Design (HLD)

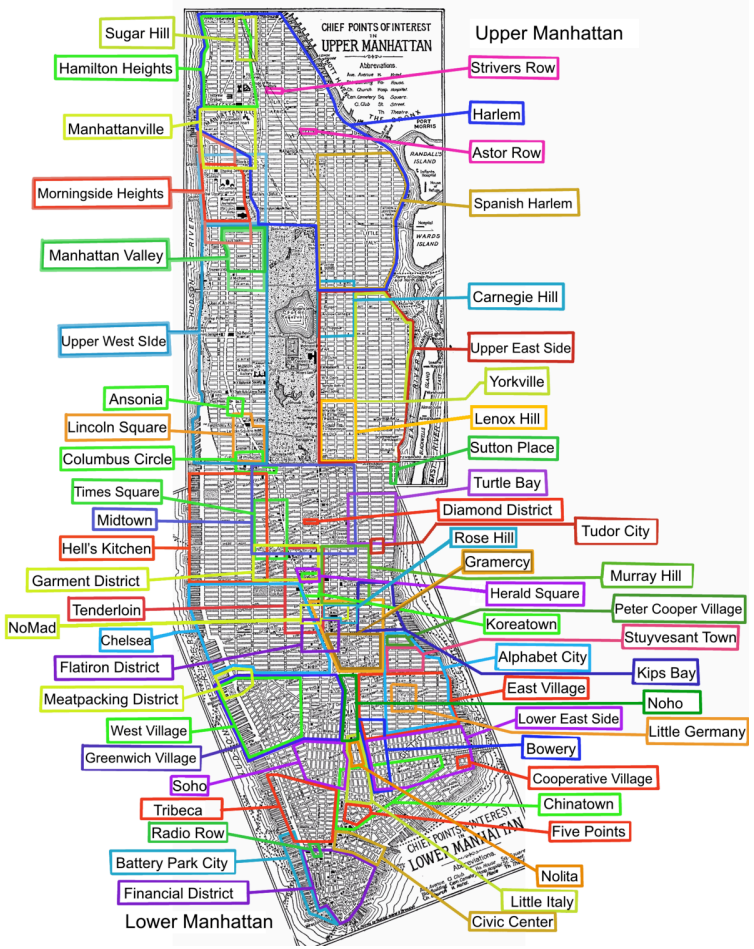
explains the architecture that would be used to develop a system. The architecture diagram provides an overview of an entire system, identifying the main components that would be developed for the product and their interfaces

As high level as possible while keeping everyone building the same thing

Perspectives on architecture

Different ways to "draw" the system will help understand it in different ways.

For most systems, there is one "best" way to show it. Sometimes, you need more.



CC BY-SA 3.0 Roke~commonswiki

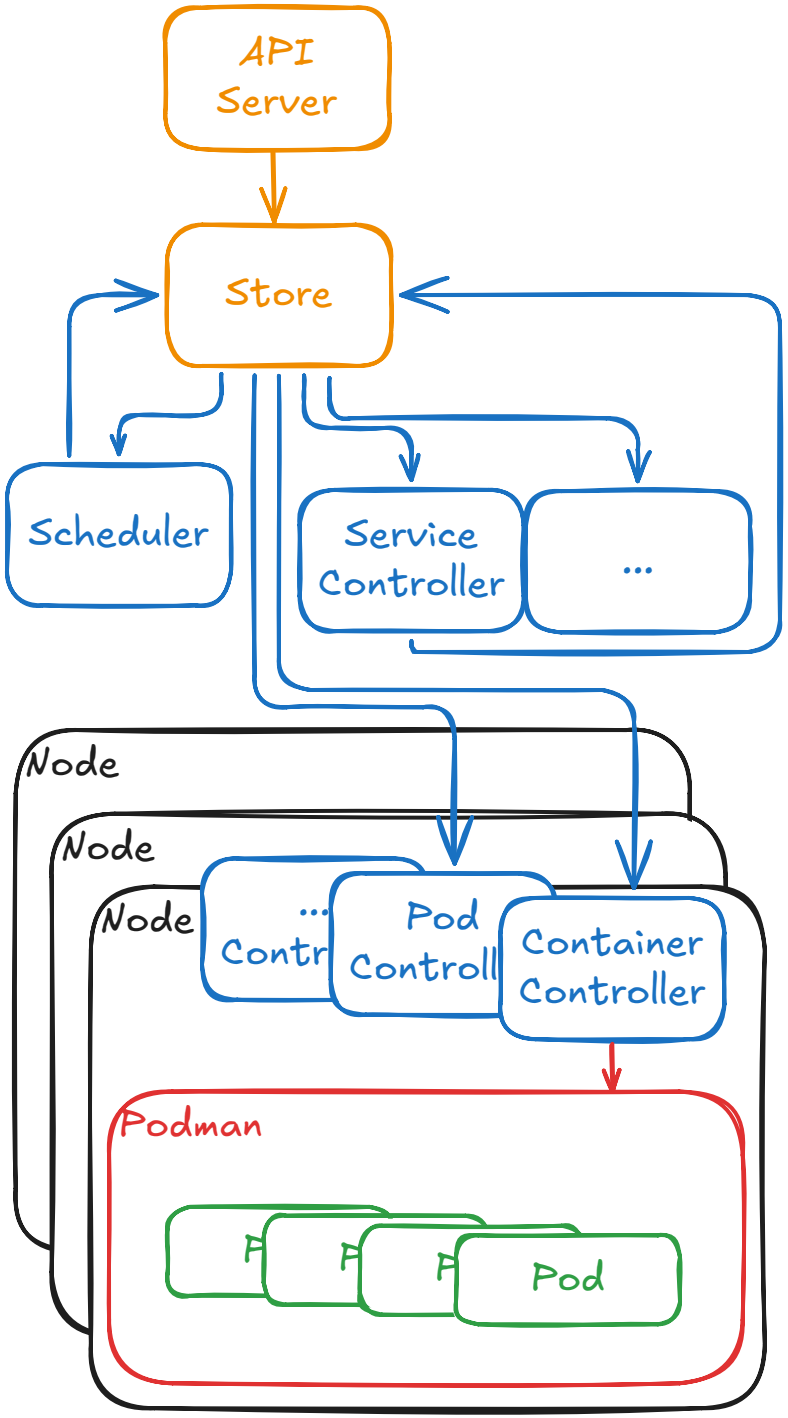


CC-BY-SA-2.0 Peterfitzgerald

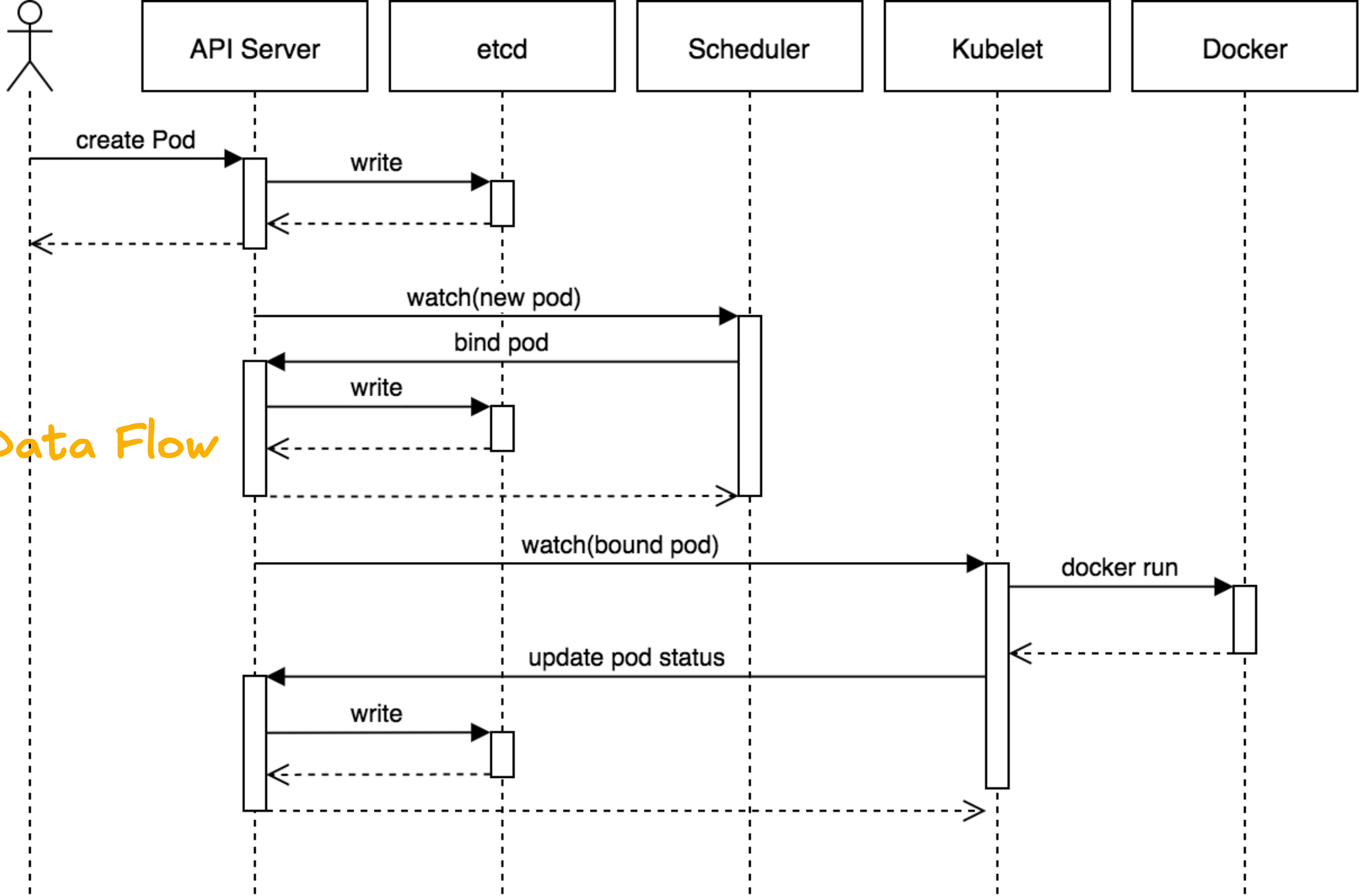
Components: What are the building blocks and what is their hierarchy?

Data flow: How does information move through the system?

Components


API Server
Store
Scheduler
Service Controller
...
Node
Node
Node
...
Contr
Pod Controll
Container Controller
Podman
Pod

Data Flow


API Server
etcd
Scheduler
Kubelet
Docker
create Pod
write
watch(new pod)
bind pod
write
watch(bound pod)
docker run
update pod status
write

Data Formats

```
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "name": "test-pod",
    "labels": {
      "app": "test"
    }
  },
  "spec": {
    "containers": [
      {
        "name": "health",
        "image": "health",
        "env": {
          "MY_ENV_VAR": "value"
        }
      }
    ]
  }
}
```

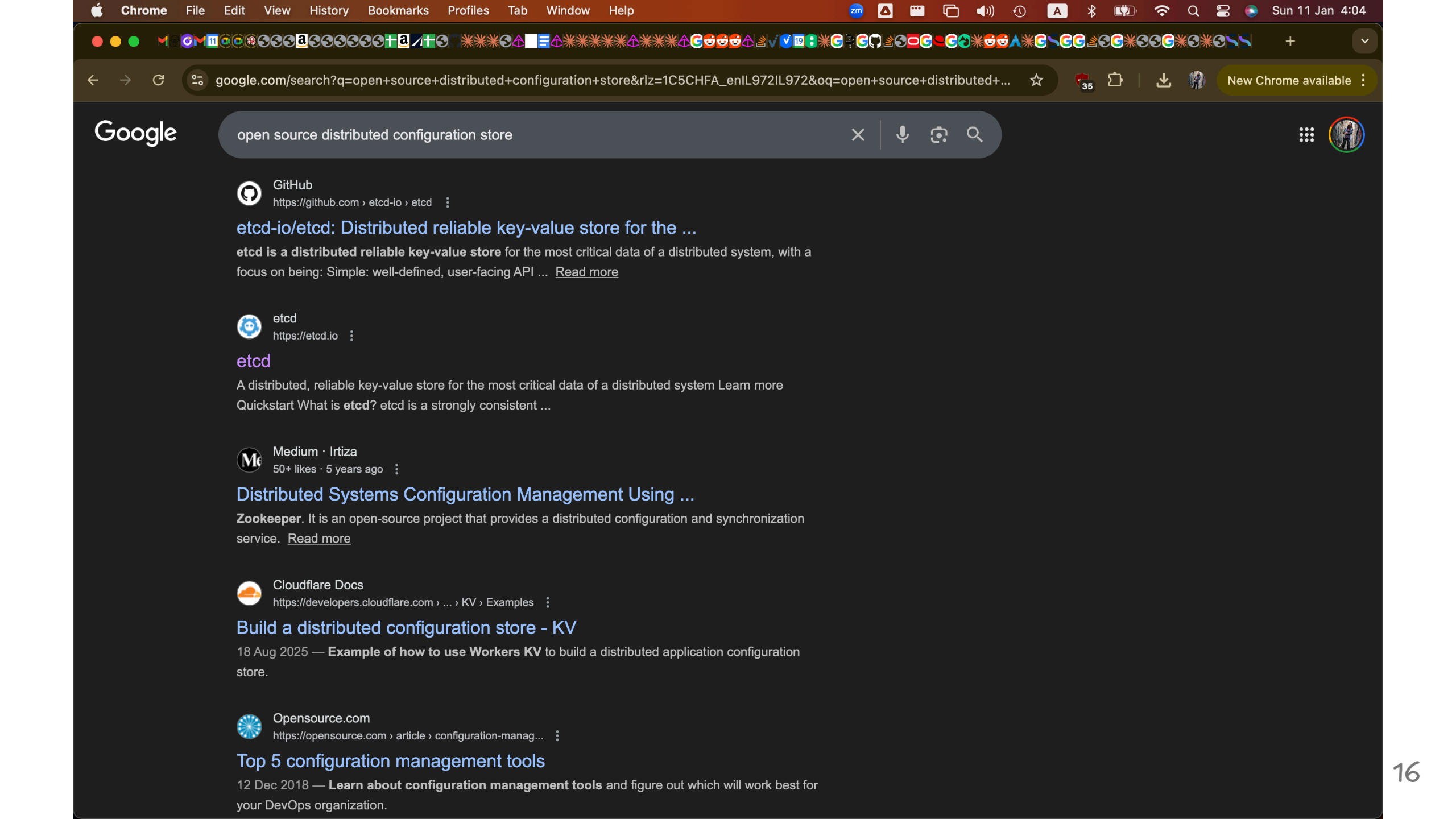


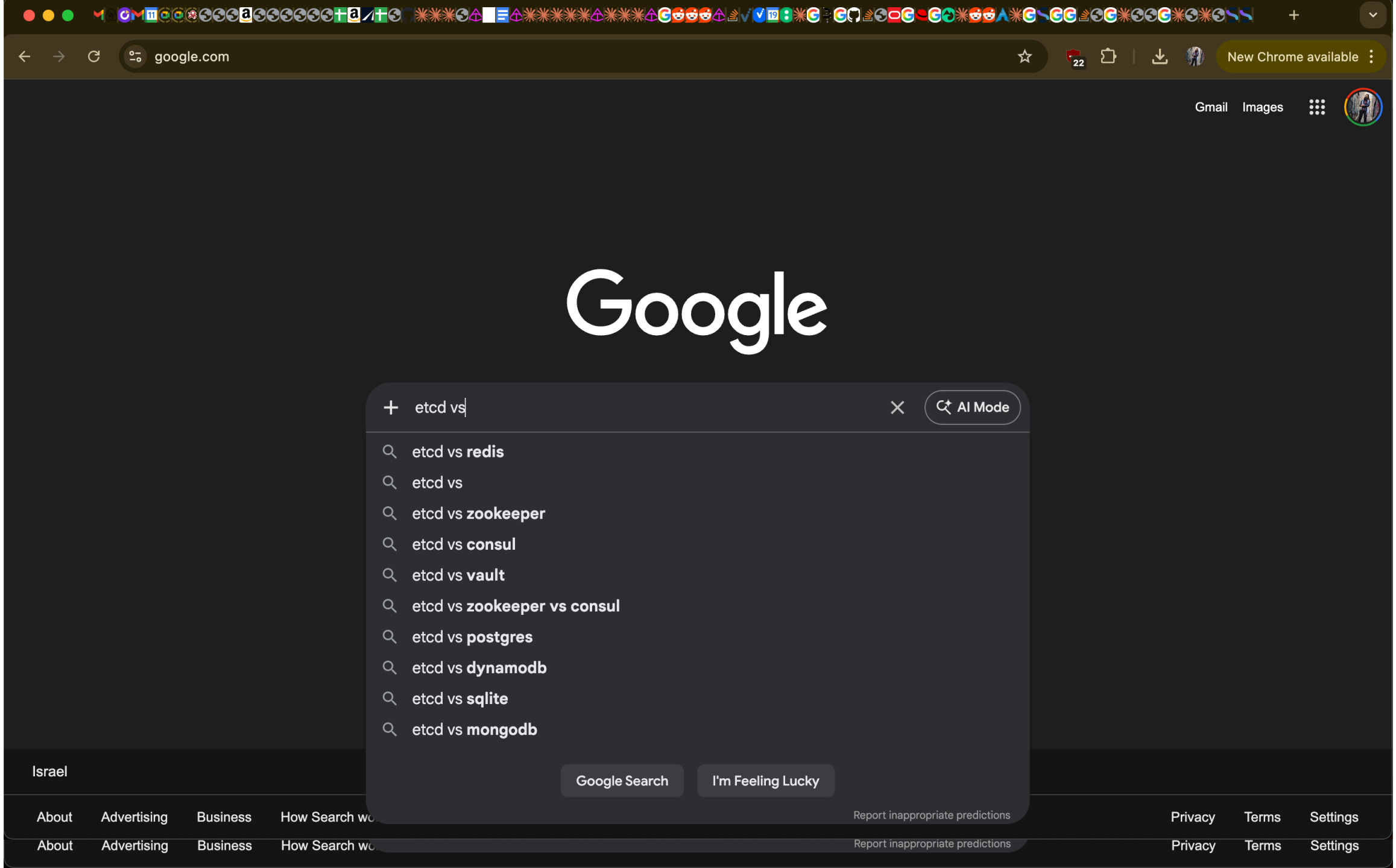
What can we build upon?

Obviously, Podman/Docker.

HTTP Framework

Distributed Config Store?


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Sun 11 Jan 4:04
google.com/search?q=open+source+distributed+configuration+store&rlz=1C5CHFA_enIL972IL972&oq=open+source+distributed+...
New Chrome available
Google
open source distributed configuration store
GitHub
https://github.com › etcd-io › etcd
etcd-io/etcd: Distributed reliable key-value store for the ...
etcd is a distributed reliable key-value store for the most critical data of a distributed system, with a focus on being: Simple: well-defined, user-facing API ... Read more
etcd
https://etcd.io
etcd
A distributed, reliable key-value store for the most critical data of a distributed system Learn more Quickstart What is etcd? etcd is a strongly consistent ...
Medium · Irtiza
50+ likes · 5 years ago
Distributed Systems Configuration Management Using ...
Zookeeper. It is an open-source project that provides a distributed configuration and synchronization service. Read more
Cloudflare Docs
https://developers.cloudflare.com › ... › KV › Examples
Build a distributed configuration store - KV
18 Aug 2025 — Example of how to use Workers KV to build a distributed application configuration store.
Opensource.com
https://opensource.com › article › configuration-manag...
Top 5 configuration management tools
12 Dec 2018 — Learn about configuration management tools and figure out which will work best for your DevOps organization.


google.com
New Chrome available
Gmail
Images
Google
etcd vs
AI Mode
etcd vs redis
etcd vs
etcd vs zookeeper
etcd vs consul
etcd vs vault
etcd vs zookeeper vs consul
etcd vs postgres
etcd vs dynamodb
etcd vs sqlite
etcd vs mongodb
Google Search
I'm Feeling Lucky
Report inappropriate predictions
Israel
About
Advertising
Business
Privacy
Terms
Settings

Which stack to use?

C++

Easy to hire

Wide support

Performant

Footguns

Slow development

Rust

Great tooling

Compiler has your back

Passionate users

Performant

Hard to hire

Go

Strong CloudTech ecosystem

Reasonable Performance

...

Python

You know it

Batteries included

Fast development

Easy to hire

Slow runtime

Our stack

Podman

Python

FastAPI

Pytest

Testing

Why?

How?

How much?

Why test our product?

Otherwise, we have no idea if it works.

And software has a very strong tendency to not do what we meant.

Why automate our tests?

Because we'll want to run them again, and again, and again.

And because if they're cheap to run, we will build better software.

If they're expensive, we'll skimp on rerunning and build a bad product.

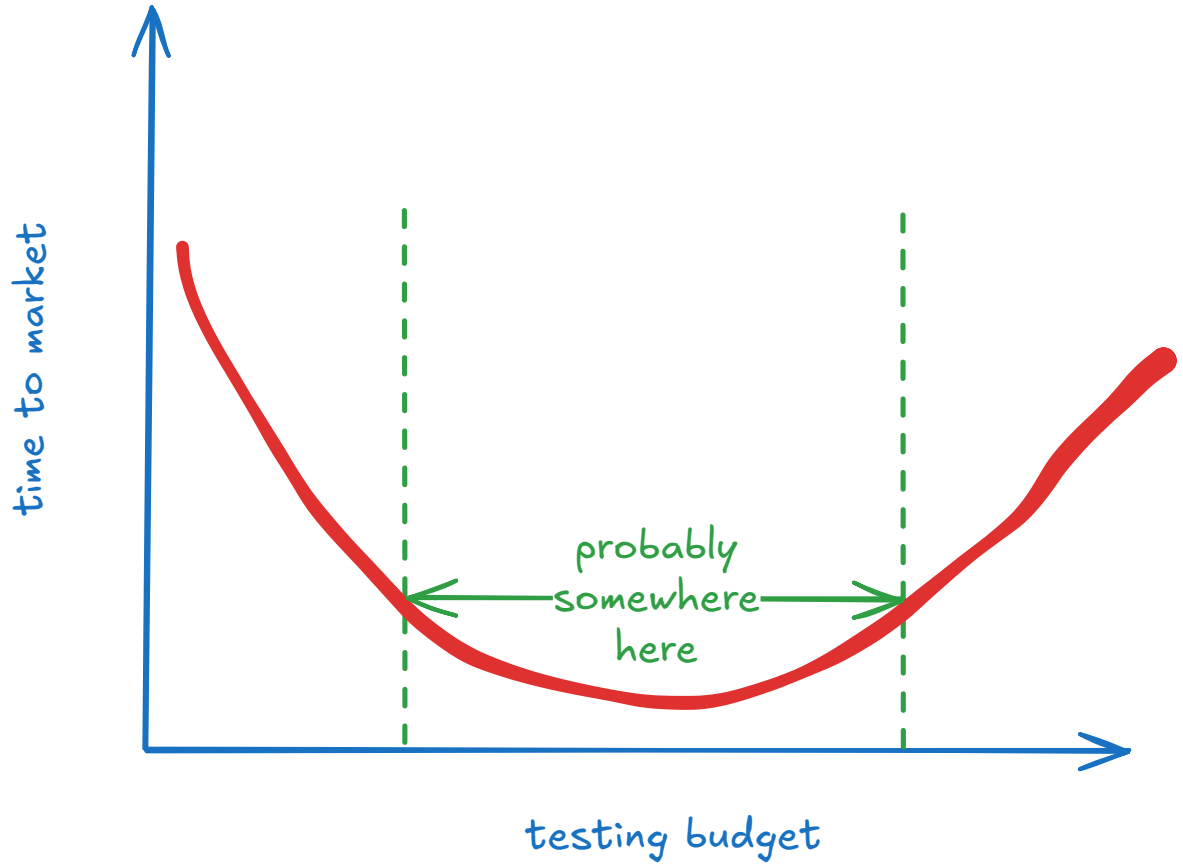
How?

This is not a tutorial on using pytest, it's very simple, you can learn it online.

An in-depth workshop on how to write good tests:
https://sonoflilit.github.io/testing/

How to choose good test cases: https://sonoflilit.github.io/testing/#12

How much should we test our product?



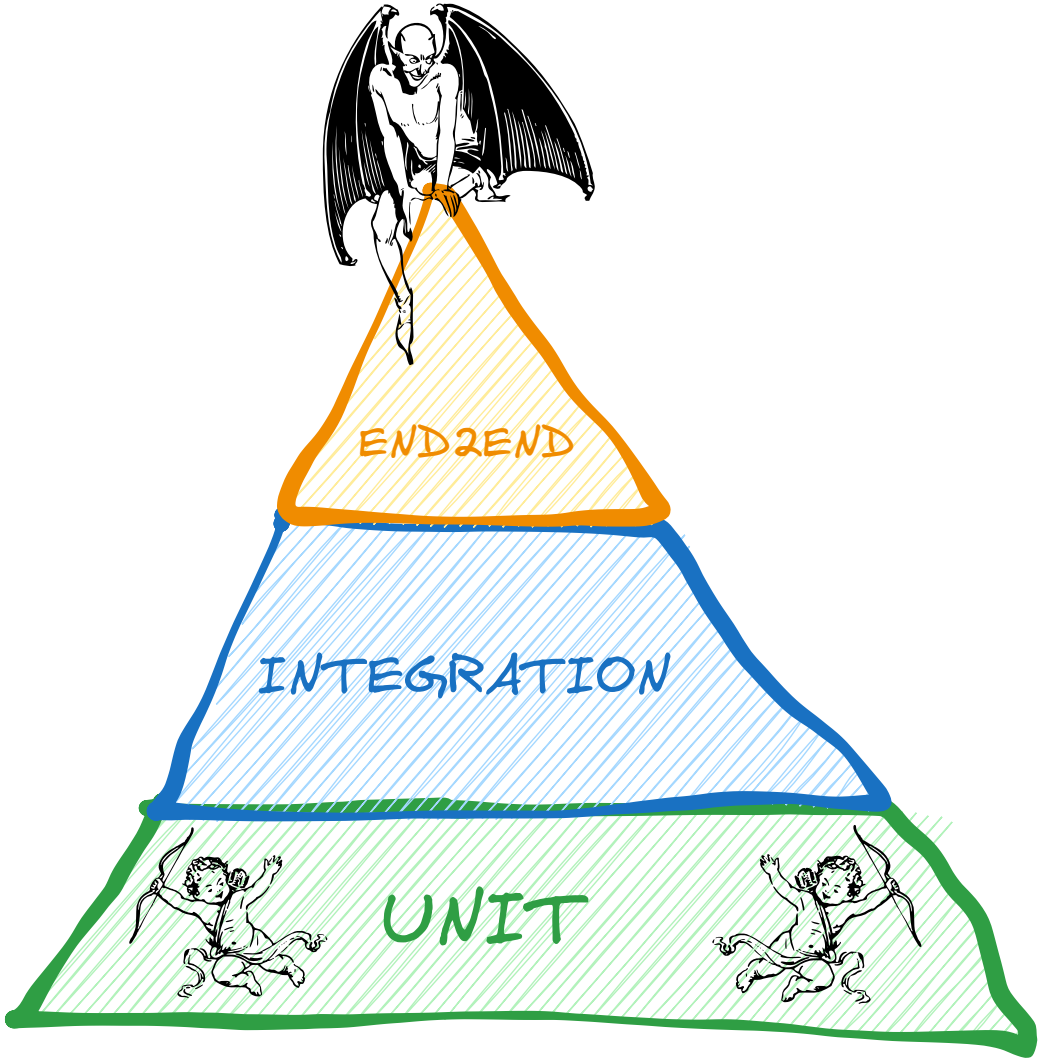
Without reading the code, what are we convinced it does?

TESTING IS RISK MANAGEMENT

TESTING IS RISK MANAGEMENT

TESTING IS RISK MANAGEMENT

The testing pyramid



Planning

Never go full waterfall

The more complex the system, the more you will need to deviate from the plan.

Software is always extremely complex.

Iteration

The more we build, the better we know what we're building. So lets build a bit, then adjust the plan, then build a bit more, then adjust...

The nearer we are, the more detailed the Roadmap

1. Simple Store for 1 resource: Pod
2. Controller for Pod that only prints
3. Controller for Pod that runs Python functions in threads
4. API for Pod
5. Another resource
6. Run Podman containers
7. More resources?
8. Use etcd?
9. Distributed control plane?

The POC

Questions?

Stay in touch! bit.ly/aurs-dojo